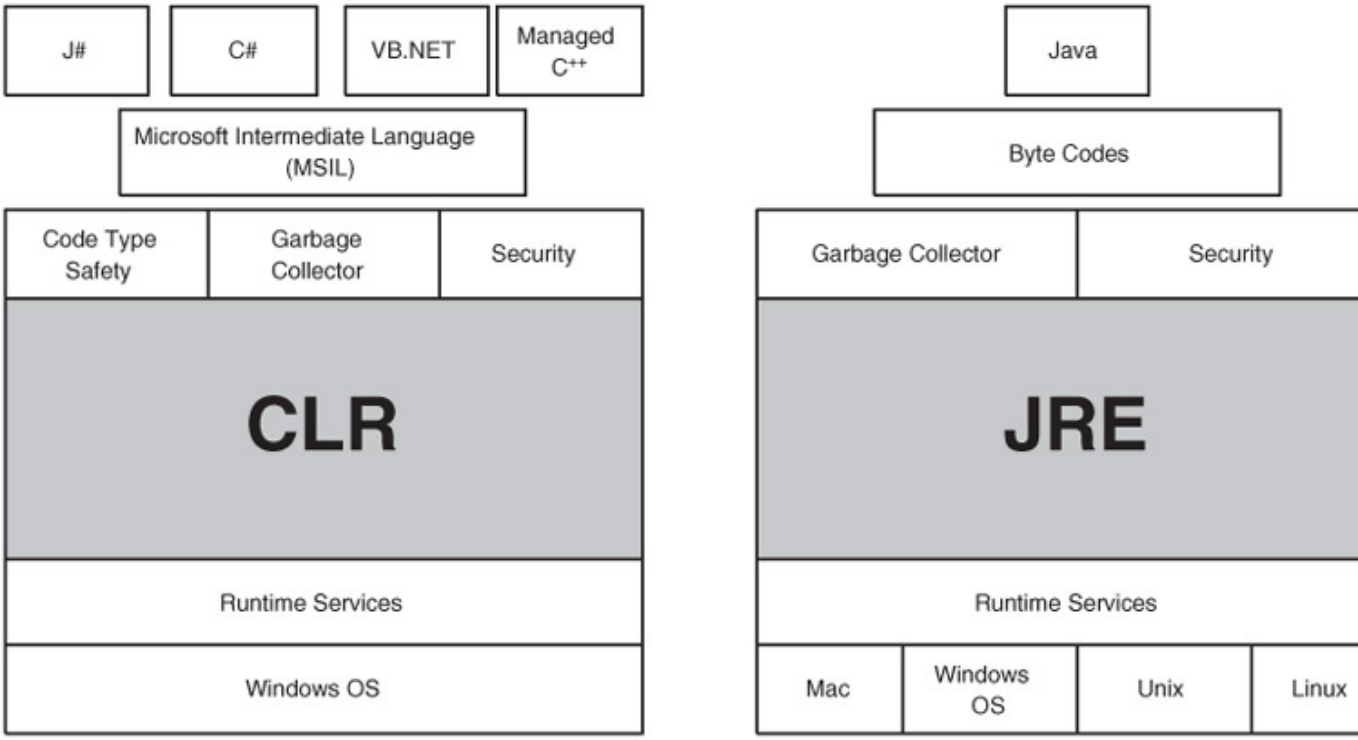
Lecture 01: Introduction to C#

Objectives:

- Learn about the .NET Framework and its relation to C#
- Understand the Structure of a C# program – namespace, classes & methods
- Learn about the two categories of data-types – value types and reference types
- Study some basic standard classes: Console, Object and String

1. The .NET Framework

The .NET Framework is a collection of Software development tools (similar to JDK) that can be used to write, debug, compile and execute programs.



It consists of :

- a collection of languages (Visual C# .NET, Visual C++ .NET, Visual Basic .NET, Visual J# .NET)
- compilers, debuggers and other tools.
- Common Language Runtime (CLR – similar to JVM)

When you write a program using C# (or any .NET language), it must first be compiled into an intermediate common language, MSIL (similar to byte codes). Although the resulting file is an EXE, it can only run on a machine that has the CLR installed.

On a machine that has CLR installed, a compiled C# program executes using a JIT. This translates the code from MSIL into machine language (similar to Java interpreter).

You can download the .NET Framework SDK (108MB) freely on this Link.
Note: You must install the .NET Framework Redistributable Package version 1.1 prior to installing the .NET Framework SDK.

Once you have the SDK installed, you can use any editor to write your C# programs. One such good editor (actually an IDE) that is free is SharpDevelop (7.2MB).

A good advantage of this IDE is that it allows you to handle both single files and projects. It also offers code completion and a limited form design features.

An alternative is to get Visual Studio .NET. This comes with the SDK and a Sophisticated IDE (6 CDs) that allows you to develop applications in C# and many other .NET languages.

An advantage here is that you have everything you would need to develop a .NET application whether in C#, Visual C++, or Visual Basic.

One disadvantage is that you must write your application as a project even if it is a simple HelloWorld example. It is also a bit complex to install.

2. Structure of a C# Program

```
using System;

namespace Greeting
{
      class HelloWorld
      {
            public static void Main(string[] args)
            {
                  Console.WriteLine("Hello World!");

            }
      }
}
```

NameSpaces:

- Classes (also Interfaces, Structs and Enums) are organized hierarchically into namespaces (packages in Java).
- The keyword, **namespace**, is used to declare a namespace and a pair of braces are used to enclosed all members of a namespace.
- The **using** keyword is used to indicate that a reference is being made to members of other namespaces. Alternative to the "**using**" keyword is to use fully qualified name, as is:

  `System.Console.WriteLine("Hello World!");`
- If a class is not enclosed in a namespace, then it is assumed to be part of a

global namespace, which has no name.

- **Note:** Unlike Java, namespaces are not in any way linked to the directory structure on the file system.

Some of the most important namespaces in the CLR that we shall be encountering are:

NameSpace	Description
System	Contains classes that implement basic functionalities like Console I/O, mathematical operations, data conversions etc.
System.IO	Contains classes used for file I/O operations.
System.Net	Contains classes that provide access to Windows network functions.
System.Net.Sockets	Contains classes the provides access to windows socket interface
System.Collections	Contains classes that implement collections of objects such as linked list, queue, hashtable etc.
System.Drawing	Contains classes that provide basic graphics functionalities.
System.Windows.Forms	Contains classes for Windows GUI applications
System.Threading	Contains classes that are used for multithreading programming.
System.Web	Classes that implement HTTP protocol to access web pages.

Classes:

As in Java, a class is declared using the **class** keyword, and all members of a class must be enclosed within a pair of braces. We discuss more about classes later.

The Main Method:

The Main method is the entry point of a C# application. Like in Java, it must be **static**. However, in C# it can take three different forms as follows:

```
public static void Main()
public static void Main( string[] args )
public static int Main( string[] args )
```

Notice that the first version takes no arguments, the last returns **int**, and the second is equivalent to what obtains in Java.

Console I/O:

The Console class of the **System** namespace provides a number of static methods for console IO. Example1 uses the **WriteLine** method to print the greeting message. There is also a **Write** method. These two methods are overloaded to take different and variable parameters. The **Console** class also provided the following methods for reading.

`public static int Read()` : reads a character
`public static string ReadLine();` : reads a line as a string

Naming Conventions:

As we can observe from example1, the naming conventions are a litle different from java naming conventions, although both languages use mixed case when an identifier involves more than one word, eg. **W**rite**L**ine.

In C#, the convention is to start the name of Namepaces, Classes, Interfaces, Structs, Enums, Properties and Methods with capital letters. Only names of variables start with small letters. Note in particular, that the **Main** method must start with a capital letter.

3. **Value Types:**
   As in Java, variables in C# are of two types, namely, value types (primitive types) and reference types. A variable of reference type holds the address of an object. A variable of value type holds an actual value. C# has more value types than Java as the following table shows.

C# Type	.Net Framework (System) type	Signed?	Bytes Occupied	Possible Values
sbyte	System.Sbyte	Yes	1	-128 to 127
short	System.Int16	Yes	2	-32768 to 32767
int	System.Int32	Yes	4	-2147483648 to 2147483647
long	System.Int64	Yes	8	-9223372036854775808 to 9223372036854775807
byte	System.Byte	No	1	0 to 255
ushort	System.Uint16	No	2	0 to 65535
uint	System.UInt32	No	4	0 to 4294967295
ulong	System.Uint64	No	8	0 to 18446744073709551615
float	System.Single	Yes	4	Approximately ±1.5 x 10^{-45} to ±3.4 x 10^{38} with 7 significant figures
double	System.Double	Yes	8	Approximately ±5.0 x 10^{-324} to ±1.7 x 10^{308} with 15 or 16 significant figures
decimal	System.Decimal	Yes	12	Approximately ±1.0 x 10^{-28} to ±7.9 x 10^{28} with 28 or 29 significant

				figures
char	System.Char	N/A	2	Any Unicode character (16 bit)
bool	System.Boolean	N/A	1 / 2	true or false

As is Java, each of these types is associated with a wrapper class that can be used to box it (or wrap it) into object.

However, unlike in java, boxing is automatic. If a value type is used where a reference type is expected, the associated reference type is automatically assumed. For example, the following is a valid statement:

```
    Object obj = 3;    //equivalent to: Object obj = new
Int32(3);
```

Unboxing however, is not implicit. We need an explicit casting to achieve it:

```
     int n = (int) obj.
```

Note that we can also call methods on value types. The method of the corresponding wrapper class is automatically called:

```
     Console.WriteLine(3.GetHashCode());
```

Example 2:

```
using System;

namespace DataTypes
{
     class ReadingData
     {
          public static void Main(string[] args)
          {
               Console.Write("Enter an integer : ");
               int i1 = int.Parse(Console.ReadLine());
               Console.Write("Enter another integer: ");
               int i2 = int.Parse(Console.ReadLine());
               Console.WriteLine("Sum: "+i1 + " + " + i2 + " = " + (i1 +
i2));

               Console.Write("Enter a double : ");
               double d1 = Double.Parse(Console.ReadLine());
               Console.Write("Enter another double: ");
               double d2 = Double.Parse(Console.ReadLine());
               Console.WriteLine("Product: {0} x {1} = {2}", d1, d2, (d1
* d2));
               Console.ReadLine();
          }
     }
}
```

Note that each wrapper class has a **Parse** method that can be used to covert a string to a corresponding value type and this method can be called either on the value type or on the wrapper class.

Note also that in a WriteLine statement, in addition to using the plus operator to concatenate many strings into one, we can use a sequence of numbers 0, 1, etc., enclosed in braces as place holders for expressions. The string is then followed by the expressions corresponding to the sequence of numbers.

4. Some Important Classes:

The object class:

In C#, every class is either directly or indirectly, a subclass of the **object** class. In fact, technically, even the value types inherit from the object class, since they exhibit the behaviors of their wrapper classes.

Notice that the object class can be spelt with small or capital **O** (Object/object).

The following are some of the methods of the object class, which all other classes inherits.

bool Equals(object) static bool Equals(object, object)	Determines whether two **Object** instances are equal.
int GetHashCode()	Serves as a hash function for a particular type, suitable for use in hashing algorithms and data structures like a hash table.
Type GetType()	Gets the Type of the current instance.
static bool ReferenceEquals(object, object)	Determines whether the specified **Object** instances are the same instance.
String ToString()	Returns a String that represents the current **Object**.

Note: As in Java, we may need to override the Equals and ToString methods in our classes.

The string class:

The **String** class in C# is almost identical to String in Java. Objects of type string are immutable, can be created with or without using new, and can be concatenated using the plus operator.

Again, note that string can be spelt with lower of upper case letter S (String/string).
The following are some of the methods of the String class. You may need to check their details in the CLR documentation.

static int Compare(s1, s2)	Overloaded. Compares two specified **String** objects.
int CompareTo(string)	Overloaded. Compares this instance with a specified object.
static string Copy(string)	Creates a new instance of **String** with the same value as a specified **String**.
bool EndsWith(string)	Determines if the end of this instance matches the specified **String**.
bool Equals(object)	Overloaded. Overridden. Determines whether two **String** objects have the same value.
CharEnumerator GetEnumerator()	Retrieves an object that can iterate through the individual characters in this instance.

int IndexOf(char) int IndexOf(string)	Overloaded. Reports the index of the first occurrence of a **String**, or one or more characters, within this instance.
String Insert(int, string)	Inserts a specified **string** at a specified **index** of this string. Returns the updated string.
int LastIndexOf(char) int LastIndexOf(string)	Overloaded. Reports the index position of the last occurrence of a specified Unicode character or **String** within this instance.
String PadLeft(int) String PadLeft(int, char)	Overloaded. Right-aligns the characters in this instance, padding on the left with spaces or a specified Unicode character for a specified total length.
String PadRight(int) String PadRight(int, char)	Overloaded. Left-aligns the characters in this string, padding on the right with spaces or a specified Unicode character, for a specified total length.
String Remove(int index, int count)	Deletes a specified number of characters from this instance beginning at a specified position.
String Replace(char, char) String Replace(string, string)	Overloaded. Replaces all occurrences of a specified Unicode character or **String** in this instance, with another specified Unicode character or **String**.
String[] Split(char[])	Overloaded. Identifies the substrings in this instance that are delimited by one or more characters specified in an array, then places the substrings into a **String** array.
bool StartsWith(string)	Determines whether the beginning of this instance matches the specified **String**.
String Substring(int start) String Substring(int start, int count)	Overloaded. Retrieves a substring from this instance.
char[] ToCharArray()	Overloaded. Copies the characters in this instance to a Unicode character array.
String ToLower()	Overloaded. Returns a copy of this **String** in lowercase.
String ToString()	Overloaded. Overridden. Converts the value of this instance to a **String**.
String ToUpper()	Overloaded. Returns a copy of this **String** in uppercase.
String Trim() String Trim(char[])	Overloaded. Removes all occurrences of a set of specified characters from the beginning and end of this instance.
String TrimEnd(char[])	Removes all occurrences of a set of characters specified in a Unicode character array from the end

	of this instance.
String TrimStart(char[])	Removes all occurrences of a set of characters specified in a Unicode character array from the beginning of this instance.

Example 4: The following example takes a full path and breaks it into various components.

```
using System;

public class FilenameProcessor {
     public static void Main(String[] args) {

          String fullName = "d:/workarea/lab02/MoveRectangle.java";
          char separator = '/';
          int dotPosition = fullName.IndexOf('.');
          int lastSlashPosition = fullName.LastIndexOf(separator);
          Console.WriteLine("The full name is: "+fullName);

          String path = fullName.Substring(0, lastSlashPosition);
          Console.WriteLine("The path is      : "+path);

          String fileName = fullName.Substring(lastSlashPosition+1,
                                    dotPosition-lastSlashPosition-1);
          Console.WriteLine("The file name is : "+fileName);

          String fileExtension = fullName.Substring(dotPosition+1);
          Console.WriteLine("The extension is : "+fileExtension);
     }
}
```

Note that the second argument in the method, **String substring(int start, int count),** is the number of characters to extract, starting from the first argument.

If your application makes frequent changes to string objects, such as insertion, and replacement of characters, then it may be more efficient to use the **StringBuilder** class (similar to StringBuffer in Java) which is mutable.